

LISTENING EDITION

ChatGPT Work and Codex Site Tools Today

A speech-friendly guide to the WebMCP features available in the ChatGPT desktop app's built-in browser

Last verified on 3 September 2026. This edition uses one-column reading order, short sections, descriptive headings, and plain language instead of dense tables or long code listings.

The short answer

This project is built specifically for ChatGPT Work and Codex Site tools in the ChatGPT desktop app's built-in browser. A WordPress page registers named JavaScript tools. ChatGPT Work or Codex can discover them, send schema-shaped arguments, and receive the structured result returned by the page.

The tools run inside the live page. They share the editor, signed-in session, selection, and unsaved state that the user is already viewing. This is a page collaboration model, not a generic backend MCP connection.

What is available now

The following statements describe OpenAI's current Site tools product, not every feature in the broader WebMCP proposal.

- Site tools are available to ChatGPT Work and Codex in the built-in browser of the latest ChatGPT desktop app when the current page provides them.
- OpenAI currently recommends GPT-5.6 Sol or GPT-5.6 Terra. GPT-5.6 Luna currently has WebMCP disabled.
- Availability depends on rollout, app version, workspace, and the current page. Site tools are not available in Enterprise or Edu workspaces.
- The user can disable Site tools under Settings, Browser, Permissions, Enable site tools.
- The built-in browser currently discovers top-level imperative JavaScript tools, not declarative form tools or tools registered inside iframes.

Do not assume this support exists in ChatGPT on the web, the Codex command line, an IDE extension, a cloud task, or an external browser. The product target is the ChatGPT desktop built-in browser.

What this WordPress plugin exposes

- Seven read tools for navigation, editor context, blocks, block types, theme design tokens, patterns, and templates.
- Eight optional write tools that stage unsaved Gutenberg changes.
- One separately gated save-post tool that persists or publishes the staged editor state.
- No REST-backed server ability catalog and no backend MCP tools.

How discovery and execution work

Stage one

The user opens wp-admin directly in the ChatGPT desktop app's built-in browser.

Stage two

JavaScript in the top-level WordPress page registers the imperative Site tools.

Stage three

ChatGPT Work or Codex receives the tool names, descriptions, and input schemas.

Stage four

The built-in browser reviews the invocation before WordPress carries it out.

Stage five

The browser invokes the registered callback inside the live page.

Stage six

The callback returns a compact result, and the agent can inspect the page to verify the outcome.

Tool lifetime

Tools belong to the document that registered them. A full navigation or reload replaces the document, invalidates the old handles, and creates a new discovery boundary. Register important tools early, wait for registration, and rediscover after navigation.

For same-document application changes, OpenAI does not publish a Site tools refresh delay or a page API that forces the desktop app to rebuild the agent's tool inventory. Keep tool names and schemas stable, then validate the current route and editor state at execution time.

Inputs, results, and errors

Use narrow JSON Schemas with useful property descriptions. A schema helps ChatGPT Work or Codex form a call, but it is not an authorization boundary. The callback must validate again before changing state.

Return structured data only after the operation has completed. A useful result says whether the action succeeded, what changed, which object was affected, and whether important state remains unsaved.

For an expected refusal, return a structured reason. For an unexpected failure, reject the callback. Do not depend on a client-specific error string.

Safety and confirmation

- Treat tool definitions and results as untrusted website content.
- Reuse WordPress authentication, authorization, validation, and editor invariants.
- Keep read tools on by default and require an administrator to enable editor writes.
- Keep save-post behind a separate gate and an in-page confirmation showing the exact arguments.
- Expire unanswered confirmations and distinguish expiry from an explicit user decline.
- Remember that the built-in browser safety review and the plugin's own gates are separate layers.

Cancellation is cooperative. Observe a callback signal before side effects and after waiting for confirmation. It can prevent work that has not started, but it cannot roll back a completed write.

Implementation checklist

- Target ChatGPT Work and Codex Site tools in the ChatGPT desktop app's built-in browser.
- Register imperative tools on `document.modelContext` in the top-level page.
- Do not put required Site tools inside iframes.
- Keep a small, stable frontend-only inventory.
- Reject `serverRegistered` ability records even when they also claim client provenance.
- Subscribe for late frontend registrations and handle registration rejection.
- Rediscover after every full navigation or reload.
- Use the live Gutenberg stores and re-read mutable state before targeted edits.
- Describe side effects accurately and return enough state to verify them.
- Test in the ChatGPT desktop built-in browser. Treat system Chrome as a deterministic page-side regression harness only.

Sources

OpenAI Site tools: <https://learn.chatgpt.com/docs/webmcp>

WebMCP specification: <https://webmachinelearning.github.io/webmcp/>

Project: <https://github.com/galatanovidiu/webmcp-adapter>